MULTI-AGENT AI FOR GAME DEVELOPMENT

BREACHPOINT

GAME DESIGN DOCUMENT — FINAL DRAFT

ASSIGNMENT	#02 — GDD Final Draft
AUTHOR	Juan Diego Lugo
DATE	29 July 2026
ENGINE	Unreal Engine 5.8 — native C++ / GAS

1. Executive Summary

1.1 Concept

Breachpoint is a **4v4 team arena first-person shooter** on the Halo sandbox model. Recharging **shields over finite health** give every fight a break-off point. A **two-weapon carry limit** plus one contested power weapon make the map itself an objective. The **golden triangle** — shoot, grenade, melee — is available in every engagement, and a **Grappleshot** turns a three-level arena into a traversal playground: pull to a ledge, yank a rocket off the floor, close on a reloading enemy.

Bots fill every empty slot through the same input path a human uses, so a match always starts.

1.2 Win and Loss Conditions

- **Win:** first team to **25 kills**, or the higher team score when the **8:00** timer expires.
- **Tiebreak:** sudden death — no respawns, first kill wins, **60-second cap**; if uncontested, higher team damage dealt wins.
- **Loss:** any other scoreline at the timer.

1.3 The Baseline Scope Rule — Breachpoint Passes

Could two developers build a rough but playable version of this game in 48 hours, without any AI? **Yes.** The UE 5.8 First Person template plus a shields attribute, two weapons, and a grapple is a game-jam-scale prototype. The core loop is **completely independent of AI** — which means it can be validated, iterated, and shipped whether or not a single model call succeeds. AI augments the pipeline that builds it; it is never load bearing for the game existing.

1.4 Platform, Format, and Production Reality

- **Engine:** UE 5.8, **pure native C++**, no Blueprint runtime logic. Combat, weapons, shields, and Grappleshot all on GAS.
- **Foundation:** the stock **First Person template** only. Everything above it is authored in-house. **No Lyra, no third-party gameplay code.**
- **Networking:** server-authoritative, **Steam listen server** (host + invite), behind `IBRServerLifecycle` so a dedicated-server/GameLift migration is a swap, not a rewrite.
- **Audio:** engine-native **MetaSounds** via GameplayCues.
- **Art:** sourced marketplace/free environment kit, weapons, FPS animation sets. **Art is sourced; all gameplay code is authored.**
- **Team & timeline:** one principal engineer plus the AI crew, **6 weeks**.
- **Exit criterion:** a playable **small complete game** published to a Steam demo depot.

1.5 One Agent, One Wow

The Wow moment: *the arena you fight in was designed by an agent.*

The **Arena Architect** is the One Wow agent — its output is the most player-visible artifact in the game, because the player spends the entire match inside it. From a one-paragraph brief it returns a complete

`arena_manifest.json`: three elevations, sightline caps, anti-camp spawn scoring, grapple-point coverage, and the rocket node — refuted by a critic agent *before a single wall is placed*, then built to spec.

Every technical decision in §3 and §4 is designed around making that moment work first.

1.6 Elevator Pitch

Halo's sandbox — shields, two weapons, grenades, and a grappling hook — as a 4v4 arena shooter, written from scratch in C++ by one engineer and an AI crew in six weeks.

2. Game Mechanics (Player-Facing Actions and Loop)

2.1 The Match Loop

1. **Spawn** with Assault Rifle + Magnum, 2 frag grenades, Grappleshot.
2. **Fight** — every engagement mixes the triangle: strip shields with fire, grenade to deny the retreat, melee to finish.
3. **Contest the rocket** — the Rocket Launcher respawns on a **90-second timer** at contested mid, countdown visible to both teams. It is why teams fight over a *place* instead of wandering.
4. **Die** → **respawn** in 5 seconds at the spawn point scored farthest from enemy pressure.
5. **Score or clock** — first to 25 or highest at 8:00 → carnage report → rematch.

2.2 The Golden Triangle

Verb	Role	Detail
Shoot	Ranged damage	Two carried weapons; swap 0.4 s
Grenade	Area denial	2 frags, physics-thrown, bounces off geometry
Melee	Finisher	70 dmg at contact; **rear melee = instant kill**

The rhythm — *shoot to break shields → grenade to cut the retreat → melee to finish* — **is** the game.

2.3 Shields Over Health

Two GAS attribute layers: **Shields (100)** recharge at 60/s beginning 2.5 s after damage; **Health (100) never regenerates**. Fights get a natural break-off point, a wounded player is a hunted player, and re-engaging becomes a decision instead of a reflex.

2.4 Weapons — three, each a distinct trade

Weapon	Source	Type	Role
Assault Rifle	Spawn	Hitscan, 600 RPM	The shield-stripper you always have
Magnum	Spawn	Hitscan, ×2 headshot	The finisher on a broken shield

Weapon	Source	Type	Role
Rocket Launcher	Map pickup, 90 s	Projectile, AoE	Power weapon; map control

The two-weapon limit makes the rocket a *trade*: take it and you drop your shield-stripper or your finisher. That trade is the sandbox.

2.5 Grappleshot — the traversal pillar

A GAS ability, 20-second cooldown, 20 m range, three uses: **pull to geometry** (verticality, escape, flank), **pull a weapon** (snatch the floor rocket from range), **pull an enemy** (close for the melee finish). The upper arena level is reachable primarily by grapple — the pillar is load-bearing for map access, not decorative.

2.6 The Arena

One symmetric arena, three elevations: **lower** (close-quarters), **mid** (contested; rocket spawn), **upper** (catwalks, grapple-primary). 2 team spawn zones + 4 scored neutral spawns; 35 m sightline cap; rocket node visible from all three levels.

2.7 Opponents — bots that play like players

Bots occupy every unfilled slot. They are **deterministic C++ state machines (StateTree + EQS)** — not language models — and they activate abilities through the **same input path a human uses**: no aimbot hooks, no privileged state, no side-channel damage. Three difficulty settings come from one authored behavior profile scaled by tuning values (§3.3 explains where those values come from).

Determinism is a design requirement: same seed + same tuning row ⇒ identical action trace, so automated soak tests reproduce exactly and no player faces an opponent the developer cannot replay.

2.8 What the Player Sees

Shield bar over health bar (top-left); weapon + ammo with the second weapon ghosted (bottom-right); grenade count and grapple cooldown ring (bottom-left); **distinct hit markers for shield hits vs flesh hits** — the sandbox is unreadable without that distinction; team score, timer, and rocket countdown (top-center); killfeed with medals (top-right).

3. AI Architecture — the Development Pipeline

Three agents. Every one produces a structured artifact a human reviews before it reaches the game. This section describes agents in the *development pipeline* — the agents that build the game — which is the subject of this course. Runtime opponents (§2.7) are classic game AI, not language models, and deliberately so.

3.1 The Pipeline Map (data → agent → JSON → engine)

```
brief / telemetry ■■■ AGENT ■■■ structured artifact ■■■ critic refutes
                                                    ■■■ human reviews
                                                    ■■■ lands in UE 5.8
                                                         (JSON manifest,
                                                          CSV DataTable)
```

Nothing an agent produces reaches the engine as free text. Every artifact is JSON or a CSV-backed DataTable row, schema-validated at the boundary.

3.2 Arena Architect — the One Wow agent

- **In one sentence:** it designs the arena the whole match happens inside.
- **Input:** a one-paragraph design brief plus the GDD's spatial constraints; on later passes, blockout screenshots.
- **Output:** `arena_manifest.json` — `{bounds, levels[], spawn_points[] {id, location, facing, scoring_hints}, grapple_points[], rocket_node, landmarks[], sightline_max_m, doubts[]}`.
- **What the player sees:** the space itself — where they can grapple, where they respawn, which sightlines they can hold, and the fact that the rocket is visible from everywhere so the contest is legible.
- **Why it's the Wow:** the manifest is diffable and reviewable *before* any geometry exists, so the critic agent can attack the layout — spawn camping, single-point grapple chokes, dead zones — at the cheapest possible stage. A design flaw dies in JSON, not in a playtest.

3.3 Tuning Curator

- **In one sentence:** it proposes every gameplay number in the game, against a schema, with evidence.
- **Input:** the combat schema, current tables, and match telemetry (bot-vs-bot soaks and human sessions, never pooled).
- **Output:** `DT_Weapons` rows, `DT_BotTuning` rows, and balance diffs — each `{table, row, column, current_value, proposed_value, evidence, expected_effect, risk, doubts[]}`.
- **What the player sees:** how lethal each weapon feels, how smart each bot difficulty plays, and a meta where no single option stays dominant for a week.
- **Restraint is part of the job:** outside a 45–55% win-rate band over ≥30 matches it proposes; inside the band it proposes **nothing**.

3.4 Spotter — the only runtime agent

- **In one sentence:** it tells the player one true thing about how they played.
- **Input:** end-of-match telemetry per player (accuracy, TTK, fights lost below 40% shields, grapple usage, medals).
- **Output:** `{coach_line}` — ≤30 words, must reference ≥1 telemetry stat; optionally `{spotter_line}` ≤18 words appended to notable killfeed events.
- **What the player sees:** a specific, earned correction on the carnage report — *"You lost 6 fights below 40% shields — break off and let them recharge."*
- **Never load-bearing:** host-side, asynchronous, ≤12 event calls + ≤8 coach calls per match, 3-second timeout, canned-line DataTable fallback shipped in the build. **Offline, the game is identical minus flavor.**

3.5 Prompt Constraints

Repeatable output requires constrained input. Every agent's prompt is built the same way:

Constraint	Rule
Role + single task	One job per agent, stated in one line. No agent has a second responsibility.
Schema-first	The exact output schema is in the prompt; the agent returns that shape or the call fails validation and retries.
Bounded vocabulary	Enumerated fields only (tone, difficulty tier, modifier) — never free-choice strings where a value feeds a system.
Allowlisted references	Agents may only cite identifiers that already exist (tag names, table rows, asset IDs). No invented keys — an unmatched name goes to `doubts[]` instead.
Uncertainty must survive	Every schema carries `doubts[]`; a value inferred rather than derived is flagged, never silently confident.
Length caps	Player-facing strings are hard-capped (coach ≤30 words, killfeed ≤18) so UI layout cannot break.
Temperature by task	Low for data curation (repeatability); moderate for the Spotter's voice only.

3.6 Guardrails — the AI Contract

The contract defines where the system may generate freely and where it must obey rules:

1. **Lore/design consistency** — every agent artifact is validated against this GDD; a critic agent adversarially refutes dangerous outputs (layouts, tuning) before a human reviews them.
2. **Format validation** — schema-checked at the boundary; malformed output is rejected and retried, never consumed.
3. **Content safety** — the Spotter is the only agent whose text reaches players, so its output passes a moderation + phrasing check server-side; anything rejected falls back to a canned line. Players never see unfiltered model output.

3.7 What is deliberately *not* an agent

- **Bots** — deterministic StateTree/EQS opponents (§2.7). An LLM in a predicted, server-authoritative shooter would be unfair and irreproducible.
- **The engineering crew** — the specialist coding agents that write the C++ are development tooling governed by contracts, not part of the game's AI architecture.

4. Technical Strategy

4.1 Engine Integration

Layer	Choice
Engine	UE 5.8, pure native C++, one runtime module `Source/Breachpoint/`

Layer	Choice
Combat	GAS — PlayerState-owned ASC, prediction keys, one damage pipeline
Input	Enhanced Input → InputTag → ASC (abilities bound by tag, not by binding code)
Movement	`UCharacterMovementComponent` subclass; grapple = root-motion source through saved moves
Netcode	Server-authoritative + client prediction, Steam listen server behind `IBRServerLifecycle`
UI	CommonUI activatable stack, MVVM ViewModels, event-driven (zero polling)
Audio	MetaSounds via GameplayCues
Agent artifacts	`arena_manifest.json` → level blockout · `DT_*.csv` → DataTables (scripted reimport) · Spotter strings → replicated to the killfeed/carnage widgets

Agent output enters the engine through **exactly two doors**: a JSON manifest a builder executes, and CSV-backed DataTables the game reads at runtime. No agent writes engine code or binary assets directly.

4.2 Token Budget

Call	Model	Calls / match	Tokens in	Tokens out	Total
Spotter (killfeed events, batched)	Claude Haiku	≤ 12	600	60	≈ 7,900
Spotter (coach, per human)	Claude Haiku	≤ 8	900	80	≈ 7,800
Per match					**≈ 15,700 ≈ $0.013**

Each match costs approximately 15,700 tokens, which means 1,000 matches costs approximately $13 at current Claude Haiku pricing. Caps are enforced server-side; every failure path falls back to canned lines.

Dev-time agents (Arena Architect, Tuning Curator) run during development only — a build cost, not a per-player cost, and bounded by how often a human asks for a proposal.

4.3 Constraints (named, with the decision each caused)

1. **Multiplayer determinism is the hard constraint.** A predicted, server-authoritative shooter cannot wait on — or be steered by — a network call to a language model. *Therefore* the Spotter only decorates the killfeed asynchronously and coaches post-match; bots are deterministic C++; canned fallbacks ship in the build.
2. **Six weeks, one principal, 100% authored gameplay code.** *Therefore* the sandbox ships at three weapons and one map, vehicles were never in scope, and the pre-declared cut order (rocket → bot settings → medals → Spotter → menus) is expected to be used, not held in reserve.

3. **API latency (1–4 s typical).** *Therefore* zero model calls in the simulation path, a 3-second timeout on flavor calls, and the factual kill feed always renders locally and instantly.
4. **Rate and context limits.** Dev-time agents run at human-review pace (a handful of proposals per day, far under any rate limit); the 200K context comfortably holds a manifest or a tuning table, so no pagination is needed at this scope.

4.4 Scope and Schedule

Shipped scope: Team Slayer 4v4 with bots filling any slot (3 difficulty settings); 1 three-level arena; 3 weapons; frag grenades; melee with rear-kill; Grappleshot; shields-over-health; scored respawns; CommonUI HUD and front end; canned medals + killfeed; Spotter with canned fallback; MetaSounds; Steam listen server + demo depot. **Nothing else.**

Wk	Milestone	Gate
1	Validation ladder, GAS core, shields, first weapon	Breaking a dummy's shields feels good
2	Grenades + melee → the full triangle	■■ **Fun gate** — commit or pivot
3	Grappleshot; arena blockout from the manifest	Grapple used offensively
4	Bots (StateTree + EQS); Team Slayer scoring	■■ **Go/no-go** — 4v4 end-to-end
5	Rocket; HUD + front end; Steam sessions	Two humans + six bots online
6	Balance, polish, Steam demo depot	Shipped

Honest estimate: ≈8.4 raw engineer-weeks compressed to ≈6.3 by delegating all content production to agents. It fits six weeks with near-zero slippage — which is why the cut order exists and why Week 4 is a formal go/no-go rather than a hope.

5. Revision & Growth

5.1 What the agent review crew flagged

The first draft went through the agent stress-test taught in S03. The crew found real, structural defects — on paper, before any code:

- **Exploit Hunter:** sudden death had **no clock** — two teams refusing to engage could stall a tied match indefinitely. Fixed: the 60-second cap with a damage-dealt tiebreak (§1.2), so a match always ends.
- **Exploit Hunter:** the Grappleshot is a predicted movement ability with an attach point — the single most cheat-prone surface in the design. A forged pull to an out-of-range target, or a rejected pull that leaves cooldown or position residue, is a teleport exploit. Fixed as process: the grapple carries its own cheat-attempt tests, and its rejection path must provably leave zero state before it lands.
- **Narrative/Consistency:** cutting the motion tracker (a scope cut) silently removed the player's primary awareness tool — the draft cut a feature without replacing its *function*. Fixed: directional footstep and

weapon audio promoted from polish to a **gameplay requirement**, plus the 35 m sightline cap; the trade is stated, not hidden.

- **Pacing & Flow:** the draft's build order put content before proof. Reordered: the golden triangle is built and **fun-tested in Week 2** — if the core loop fails there, the project stops while stopping is cheap.

5.2 What humans caught

A non-developer read-through was asked S03's three questions. *"Can you tell me what the player does?"* — yes, cleanly. *"Does this sound fun?"* — yes, with one hesitation: *"how do I know the rocket matters?"* That produced two changes: the rocket's respawn countdown is visible to **both teams** (the contest is announced, not discovered), and the arena constraint that the rocket node be visible from all three levels (§2.6).

5.3 The finding that outranked all of them — scope

The stress-test's biggest catch was not a rule; it was the budget. The full sandbox concept — five weapons, motion tracker, dedicated servers on day one, three distinct bot behavior trees, two maps — was **costed line by line**, every system marked *sourced* or *authored*, with only authored work consuming the schedule:

	Build inventory	Fits 6 weeks?
Full concept (first draft)	12.9 engineer-weeks	No — ~2× over
Scoped slice (this document)	**8.4 raw → 6.3 compressed**	**Yes, with the cut order**

The revision: cut breadth, keep identity. Everything that makes it the game survived — shields-over-health, the golden triangle, the Grappleshot, real bots — and everything that was breadth moved to a named Phase-2 backlog (motion tracker, plasma weapon, dedicated servers, more maps). A **pre-declared cut order** (rocket → bot settings → medals → Spotter → menus) makes the remaining risk executable instead of aspirational. Debugging this on paper cost days of documents; it would have cost the entire schedule in code.

5.4 Architecture revisions driven by the course material

- **Six agents → three.** The draft named Arena Architect, Bot Trainer, Combat QA, Balance Analyst, Weapon Curator and Spotter — over the "5+ agent types for a solo developer" red flag. Bot Trainer, Weapon Curator and Balance Analyst all did the same *kind* of work (propose table rows against a schema with evidence) and merged into one **Tuning Curator**; Combat QA was never a content agent — it is automated testing and moved to §4. Each surviving agent now does one thing exceptionally well.
- **A One Wow agent was designated** (§1.5). The draft had no single AI-driven moment; the Arena Architect is now named as it, and §3 is built around making that moment work.
- **The AI story was refocused on the pipeline.** The draft led with bots as the headline AI feature. Bots are classic game AI — they belong in mechanics (§2.7). The agents that *build the game* are the architecture.
- **A content-safety guardrail was added** (§3.6). The Spotter is the only agent whose text reaches players and had no moderation layer — a real hole in a shipping product.

- **A Prompt Constraints section was added** (§3.5) — output schemas were specified, but the prompt-side rules that make outputs repeatable were not.

5.5 What I held against feedback

The one-map, three-weapon scope (breadth is what kills six-week projects); the Spotter's deliberately non-load-bearing role (a fair multiplayer game cannot let a model decide anything); and deterministic bots over LLM opponents (irreproducible opponents are unfair and untestable). Restraint is a decision too.

Appendix A — Combat Tuning

Parameter	Value
Shields / Health	100 / 100
Shield recharge	60/s, begins 2.5 s after last damage
Health regeneration	none
Assault Rifle	8 dmg/shot, 600 RPM, 32 mag, hitscan
Magnum	22 dmg, ×2 headshot, 8 mag, hitscan
Rocket Launcher	120 dmg, radius 4 m, 2 shots, 90 s respawn
Frag grenade	90 dmg centre, radius 5 m, 2 carried
Melee	70 dmg; rear = instant kill
Grappleshot	20 s cooldown, 20 m range
Sprint	+20% speed, no cost; ends on fire/melee/grenade
Respawn	5 s, scored spawn
Match	25 kills or 8:00; sudden death 60 s cap
Arena sightline cap	35 m

Appendix B — Bot Difficulty (one profile, three scalars)

Setting	Accuracy	Reaction	Grenade use
Recruit	25%	500 ms	Rare
Marine *(default)*	45%	320 ms	Situational
Veteran	65%	220 ms	Tactical

All settings drive the same StateTree and EQS queries; only `DT_BotTuning` scalars change. No setting may drop `reaction_ms` below 200 — a superhuman guard enforced by the schema.

Appendix C — Telemetry Schema

`FBRMatchTelemetry` (per player, per match): kills, deaths, assists, accuracy per weapon, TTK distribution, shield-break→kill conversion, grenade and melee kills (front/rear), grapple uses and grapple kills, rocket holds, **fights lost below 40% shields**, medals, time alive. Consumed by the Spotter (coach lines) and the Tuning Curator (win-rate bands, regression baselines).